

# 1 Flirds 체크포인트 01 — 알고리즘 + 노벨티

## 01 · 알고리즘 + 노벨티

코드( `codes/flirds/core/flirds_estimator.py` )를 직접 읽고 검증한 알고리즘 정확본 + 노벨티 주장(각 근거 첨부). 태그:

**CODE** = 코드로 확인 / **DOC** = 문서(plan/flirds.md)에만 서술 / **PDF** = 원문 대조.

### 1.1 우리 알고리즘이 코드상 정확히 어떻게 도는가

#### 입력

estimator는 model-val-task를 전혀 보지 않는다. 입력은 오직 ( `flirds_estimator.py:6-8` , `:65-66` ):

- `logs = [(w_r, deltas_map)]` — 열린 FedAvg 레직. `deltas_map[c] = (Δw_c, n_c)` [CODE `:87-89`]
- `loss_fn(params, buffers) → scalar` — backend가 만든 val-loss closure ( `backends/{cnn,llm}.py` )
- `pkeys` — 학습가능 param 이름들 (각  $w_r$ 를 `params=Taylor` 변수 / `buffers=고정` 으로 분리) [CODE `:101-102`]

#### 핵심 계산 (round 루프, `flirds_estimator.py:97-131` )

각 round ( `w_r, dm` ) 에 대해:

1. 참여 cohort `players = dm.keys()` , per-round FedAvg weight  $pr[k] = n_k / \sum_{j \in \text{players}} n_j$  [CODE `:98-100`]
2. round aggregate  $\Delta W^r = \sum_{k \in \text{players}} pr[k] \cdot \Delta w_k$  [CODE `:109`]
3. 1 HVP: `g, u = jvp(grad(vloss), (params,), (dW,))` →  $g^r = \nabla \text{val-loss}$ ,  $u^r = H^r \Delta W^r$  [CODE `:111`]
4. 각 client:  $\phi_k \leftarrow pr[k] \cdot \langle g^r, \Delta w_k \rangle + \frac{1}{2} \cdot pr[k] \cdot \langle \Delta w_k, u^r \rangle$  [CODE `:128-131`]

결과:  $\phi_k = \sum_r pr_k^r [\langle g^r, \Delta w_k \rangle + \frac{1}{2} \langle \Delta w_k, u^r \rangle]$  (docstring `:13` ). [CODE 전체 검증]

#### Non-obvious design points (모두 **CODE** )

- round당 정확히 1 HVP. 2차 quadratic-Shapley가  $u^r = H^r \Delta W^r$  하나 + `|P_r|` 개 내적으로 붕괴 ( `:26-28` ). 이게 비용의 핵심 — in-run oracle은 round당  $2^N$  forward.
- forward HVP  $H \cdot v$  만 사용,  $H^{-1}$  절대 안 씀. `jvp(grad(.))` = forward-over-reverse AD ( `torch.func` , `:39, :55, :111` ). → influence-function의 iHVP-inversion 비용/불안정성 회피.
- true Hessian (GGN/Fisher 아님). `jvpograd` 는 진짜 Hessian-vector product. GGN은 테스트 후 기각 [DOC `flirds.md:34` ; raw `2026-06-03-phase05-estimator.md:48` ].
- per-round participant weight → partial participation(cross-device)서 정확, full participation(cross-silo)서 고정 global  $n_k / \sum n$ 으로 환원 ( `:19-24` ).
- participation normalization 없음 (locked): 값이 참여횟수에 비례 → tier 내 rank로 품질 회수 ( `:22-24` ).
- sign: client가 val-loss를 내리면  $\phi_k < 0$  ( `:28` ). selection은 가장 낮은  $\phi$ 를 keep ( `phase1_clean_run.py:88-90` ).
- free-rider  $\phi =$  정확히 0:  $\Delta w_k = 0$  → 모든 내적 0 →  $\phi_k = 0$  [CODE; 실측 `metrics.json` client 1 = `0.0` 매 seed].
- LLM val 청킹 ( `loss_chunks` ): val을 도메인별 청크로 쪼개 weighted-sum → full-val grad/HVP와 정확히 동일(선형), peak memory=1 청크. eager-attention HVP가 val=1000서 OOM 안 나게 ( `flirds_estimator.py:42-62` , `backends/llm.py:35-51` ).
- fp32 (protocol 1): utility=loss차  $\sim 1e-3 < \text{bf16}$  정밀도  $\sim 8e-3$  → bf16 불가 ( `flirds_estimator.py:34` , `backends/llm.py:12-15` ).

#### LLM backend의 3 musts (non-obvious; CNN은 안 걸림) [CODE `backends/llm.py:14-68` ]

1. `attn_implementation="eager"` — SDPA/flash는 forward-mode AD 미구현 → `jvpograd` HVP 에러 ( `:14-20` ).
2. FL state를 `named_parameters()` 키로 ( `...lora_A.default.weight` ), `get_peft_model_state_dict` 아님; `load_state_dict(strict=False)` 동기화 ( `:4-6, :69` ).

3. `get_input_embeddings()._forward_hooks.clear()` + `use_cache=False` — SFTTrainer의 grad-checkpoint hook이 functorch transform 안에서 금지 (:56-68).

## retrain/in-run oracle와의 관계

- in-run oracle** (`oracle/in_run_sv.py`): estimator가 근사하는 대상. coalition별  $U_{(b)}(S) = \sum_r [\ell(w^r + \sum_{k \in S \cap P_r} p_k^* \Delta w_k) - \ell(w^r)]$  를 exact  $2^N$  enumeration으로 Shapley화 (`in_run_sv.py:3-6, :71-122`). estimator는 이걸 Taylor로 1 HVP 근사. cross-device는 `in_run_shapley_perround` = round별  $2^{1P}$  분해 =  $2^N$  과 수학적 동일( $\Delta\phi \approx 3e-16$ , smoke `phase2_crossdevice_oracle_smoke.py` 로 증명) [CODE](#).
- retrain oracle** (`oracle/exact_sv_llm.py`): coalition마다 FedAvg 재학습 → 배포모델 점수. utility = `-val-loss` (주, 검증 대상) = (b)-estimator와 same game → Spearman +1.000으로 Shapley 계산 검증; **ROUGE-L (보조)** = retrain oracle만 썰 수 있는 배포-현실 지표지만 다른 게임(non-diff, answer\_swap 포맷에 속음) → 관찰용, divergence 자체가 val-loss utility 근거 `:42-44, :85-89` → [03](#) + [02](#).

## 1.2 기존 연구의 한계 → 우리 차별점 → 노벨티

### 우리가 직접 마주하는 한계들 (각 PDF/소스 근거)

| 선행 한계                                                   | 근거                                                                                                                              | Flirds의 응답                                                           |
|---------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| 재학습은 LLM-FL서 불가능<br>(Data Shapley는 CIFAR도 MC 필요)        | Ghorbani-Zou <a href="#">1904.02868v2.pdf</a><br><a href="#">PDF</a> ; retrain oracle N=10=2-5일/1-GPU                           | in-run(재학습 0): in-run oracle/estimator는 round당 forward 만             |
| IRDS는 centralized-per-step-data-point                   | IRDS extract <a href="#">Data Shapley in One Training Run.md</a> <a href="#">PDF</a>                                            | FL per-round-client-LoRA로 lift                                       |
| IF는 iHVP collapse + 비수렴<br>→ LLM서 약함                    | "Do IF Work on LLMs?" <a href="#">2409.19998</a> ; Basu fragile <a href="#">2006.14651</a> [DOC <a href="#">flirds.md:248</a> ] | forward HVP $H \cdot \Delta w$ ( $H^{-1}$ 안 씀) + in-run → 두 원인 모두 회피 |
| FL-Shapley는 통신/서버연산 큼                                   | GTG sub-model 재구성, FedSV $O(Tm^2)$ util, ComFedSV "Everyone-Being-Heard" 라운드                                                    | 통신 0 (vanilla FedAvg의 deltas_map 만 사용) [CODE estimator 입력=logs 뿐]    |
| FL-Shapley는 non-IID서 무너지고 rare("maverick") client를 과소평가 | mavericks <a href="#">2106.10734</a> , volatility <a href="#">2405.08044</a> <a href="#">DOC</a>                                | in-run exact oracle로 안정화; 한계는 측정(검출 AUROC)                           |
| 검출기는 noise/OOD/malicious를 다 "anomaly→discard"로 뭉갠       | <a href="#">threads/noise-ood-malicious-client-separation</a> <a href="#">DOC</a>                                               | signed value로 분리 시/도; 못 푸는 부분은 특성화된 한계로 보고                           |

### 차별점 / 노벨티 주장 (각 grounding)

- 노벨티는 "최초 federated in-run"이 아니라 **교집합** [DOC [flirds.md:241-243](#)]. Ripple Shapley(AAAI'26)가 이미 federated in-run을 점유. 주장하는 비점유 교집합 = **client-level** + **in-run** + **closed-form 1st/2nd Taylor** + **HVP 상호작용항** + **zero-extra-comm** + **LoRA/LLM**. 근거=내부 4-agent + 외부 GPT 서베이(서술적; 코드 검증 아님).
- Ripple 대비**: Ripple=sample-level Jacobian propagation(eigsh, 재귀 chain) → 우리는 client-level closed-form Taylor, LoRA 명시, 2차 client-interaction항 명시 [DOC [flirds.md:235-239](#)]. 실측: Ripple ~4515s = Flirds(107s)의 ~42x, 검출도 가장 약함(noisy AUROC 0.50±0.20) [ [B](#) [raw/.../2026-06-06-sv-baseline-port-and-results.md](#) ].
- FedIF 대비**: FedIF(2025)=1st-order TraIn on  $\Delta w$ , CNN-only, aggregation 변경 → 우리는 2차 추가, LLM/LoRA, vanilla FedAvg 위 post-hoc valuation [DOC [flirds.md:245](#)].

4. **2차항이 FL에서 비로소 의미** **DOC**: IRDS Appx E.2.2는 centralized per-step(작은  $\eta$ )서 2차가 거의 무의미하다 함. FL per-round multi-step  $\Delta w$ 는 크므로 2차가 non-trivial. CNN 실측이 뒷받침: plain SGD서 1st+2nd Spearman 0.962 > 1st 0.924; **momentum 커면 역전**(1st 0.81 > 1st+2nd 0.73) → 그래서 plain SGD 고정 [ **b** raw **2026-06-03-phase05-estimator.md:75-80**; **codes/CLAUDE.md §5** ].
5. **속도/정확도 프론티어 지배** **b**:  $N=5$  near-additive → Shapley linearity로 in-run utility 쓰는 모든 방법이 같은 ranking(+1.000) → Flirds가 **5-15x 더 싸게** 같은 답 (Flirds-1st 35s vs GTG/FedSV/oracle ~530s). 큰  $N$ -강한 상호작용서 분리력은 real grid로 실측·영속화 완료 **b**: 22/25 셀 **runs/phase2\_matrix/rundirs** metrics.json (잔여 3셀=dev\_a0.5 {noisy,frand,frzero}).  $N=100$  cross-device 실측: Flirds AUROC free-rider 1.000(측정 전  $\alpha=0.0/0.01/0.1/5.0$ ), noisy 0.575~0.774( $\alpha$ 별), poison( $\alpha=0.0/0.5$ ) 1.000±0.000. 2차항 분리력도 실측 grounding 확보 — dev\_a0.5\_poison( $N=100$ ): Flirds 1.000±0.000 vs Flirds-1st 0.670±0.467; silo5\_poison( $N=5$ ): Flirds 0.917±0.118 vs Flirds-1st 0.000(완전 회피).
6. **free-rider  $\varphi$  정확히 0** [CODE+ **b**]: Flirds/in-run oracle/Banzhaf/loss-heur는 정확 0; GTG/FedSV는 within-subset renorm 희석으로  $\neq 0$  → 우리 utility 정의의 깔끔함 입증.
7. **cross-domain valuation fairness hook** [CODE **backends/11m.py:83-88**]: per-domain macro-norm 옵션 (token-norm vs domain-macro). LESS/FedDQC 등은 magnitude만 부분완화, cross-domain 비교가능성은 미해결이라는 주장 **DOC**.

**솔직한 경계**: 위 노벨티 중 1·2·3·7의 "비점유/미해결" 주장은 *서술적 서베이* 근거(코드로 증명 불가). 4·5·6은 실측 grounding 있음(5·6의 큰  $N$  분리력도 real grid 22/25 셀 영속화로 **b** — 수치는 위 5번 참조).

→ 알고리즘이 baseline들과 수식 차원에서 어떻게 다른지는 [03-baselines-and-prior-work](#).